

Infrastructure as Intent (IAI)

Whitepaper III: FinOps Deep Dive

Third in a three-part whitepaper series on Infrastructure as Intent

Elaia Raj

2026

Abstract

Cloud made it easy to spend money and hard to see what the money bought. Most organizations still learn what a change costs after it has run, when the invoice arrives. By then the decision is made and the money is gone.

The first paper in this series introduced Infrastructure as Intent, or IAI: a reasoning layer above existing infrastructure tools. A human states an outcome in plain language. An agent reads a declared source of truth, picks the right execution engines, generates the artefacts, checks the change for correctness, security, and cost, and asks for approval before anything is applied. The second paper took the security gate in depth. This paper takes the cost gate.

The argument is that cost should be decided before a change is applied, not discovered after. Quality shifted left under testing. Security shifted left under secure development. Cost has been the slowest to move. IAI moves it: every change is priced before it is applied, checked against limits written as policy, and shown to the approver in the same summary that covers correctness and security. When no change is pending, the same agent watches live spend, catches a runaway before it grows, and proposes the fix.

This is where the FinOps field is already heading: away from chasing waste after the fact, toward governance, forecasting, and tying spend to business value [1]. IAI supports that by making cost a decision at the point of provisioning, and by recording, for every resource, what it is for — so spend can be traced back to the outcome it serves.

1. Introduction and Thesis

A responsible change has to clear three questions: is it correct, is it secure, and can we afford it. The first two are answered before the change is applied. The third, in most places, is left to the monthly bill. Two of the three are decided in advance; the third is found out later.

This is a habit, not a necessity. Cost does not need a resource to run before it can be estimated. The price of a resource follows from what it is — its type, size, region, and expected use — and can be worked out before anything is provisioned. What has been missing is a place in the process where that estimate is treated as seriously as a failed security check. IAI adds that place. The cost is estimated before apply, checked against policy, and made a condition of approval.

This changes what can be done about cost. When cost is found after the fact, it can only be optimized — waste is removed after it has been paid for. When cost is decided before the fact, it can be governed — shaped at the moment of commitment, before money is spent. **In the era of the intent layer, cost is not something to optimize after the fact, but a limit to meet before the commitment.**

2. Where the Discipline Stands

FinOps has matured, and IAI lines up with the direction it has taken.

Start with the scale of the waste. Reports published in 2026 put wasted cloud spend at roughly a quarter to a third of the total — resources that are oversized, idle, forgotten, or misconfigured — with one widely cited figure above one hundred billion dollars [2][3]. At that scale, waste is a structural problem, not a housekeeping one.

The easy savings are mostly taken. For years the work was switching off idle servers, rightsizing instances, and cleaning up tags. Those were one-time gains, and practitioners now report diminishing returns: the big rocks are gone, and what remains is a high volume of smaller, harder opportunities [1]. So the field has shifted its weight. The FinOps Foundation describes the work in three phases — inform, optimize, operate — and the priority is moving from optimize toward inform and operate: understand the spend, then govern it [5]. In the 2026 survey, governance ranks among the top priorities, on the recognition that chasing waste alone does not scale without policy to prevent it [1].

Underneath this sits allocation. To manage cost per unit — per product, per team, per customer — every dollar has to be tied to the thing it serves. That tie usually depends on tags applied by hand, which are often missing or wrong: in many organizations more than a fifth of cloud spend is untagged, and attribution gaps are widening [4]. So allocation stays hard. The problem is also widening. FinOps now covers more than cloud — SaaS, licensing, and AI among them — and AI raises the stakes. A poor GPU sizing choice can cost many times what the same mistake on ordinary compute would, and usage-driven workloads are harder to forecast [6]. AI cost management is now a priority for almost every organization surveyed, up sharply in a single year [1][6].

What the field now agrees:

- Preventing waste beats chasing it after the fact.
- Spend has to be tied to business value, which needs reliable allocation.
- Cost should be owned by the teams that create it, not only by finance.
- Usage-driven workloads, AI especially, make early cost control more important, not less.

IAI addresses these at the root. It makes cost a decision at provisioning, and it records why each resource exists, so spend can be traced to the outcome it serves.

3. The Invoice Is Too Late

The core flaw in the old model is timing: the first solid signal of what a decision costs arrives after the decision can no longer be undone.

Take a common mistake. An engineer provisions a resource — an oversized database, a duplicate environment, a logging setup that stores far more than anyone reads. It is correct. It passes review. It is secure. Nothing looks at its cost, because cost is not part of the workflow. The resource runs

and bills quietly until the invoice arrives, someone investigates, and it is corrected. By then the money is gone. It was committed the moment the resource was created.

The worse version is the accidental overrun — the bill shock, where the cost is not a little high but many times what was intended: a process that creates resources in a loop, a test environment sized like production, an AI job that scales with no ceiling. Against this the old model has two tools, and both act too late:

- The budget alert fires after spend has started. It reports a loss; it does not prevent one.
- The periodic review finds savings for next time, which does not help the money already spent.

Neither sits between the decision and the commitment. The problem is not effort. It is the lack of a check at the one moment that matters — before the resource is created.

4. The Cost Gate: Decision Before Commitment

The first paper described a cost gate that estimates a change's cost before apply. Placed there, it becomes the main tool for governing cost.

When the agent generates a change, it works out the cost from the specification before anything runs. That estimate is a gate, checked on every change alongside correctness and security, and folded into the one plain summary the human approves. The approver does not sign off and learn the cost later. The cost is known, stated, and weighed first.

Written as policy, this lets the organization set cost rules and have them enforced every time: a limit on the recurring cost a change may add, ceilings that fit each environment, allowed instance types. This is the cost side of policy as code — budgets, allocation rules, and limits kept as version-controlled rules rather than a spreadsheet someone checks now and then. The wider field is moving the same way: cost guardrails that fail a deployment when it exceeds a threshold or uses a non-approved instance type, enforced like a security check [7]. Who may commit large spend can be limited the same way, on a least-privilege basis, so an expensive resource is not created on a whim.

	Conventional	IAI
When cost is known	At month-end	Before the change is applied
What you can do about it	Optimize — recover waste already paid	Govern — prevent the waste
Where the cost rules live	A spreadsheet or a review meeting	Policy as code, enforced on every change
Who sees the cost, and when	Finance, at month-end	The approver, at the moment of decision

The gate does not price everything, and it should not pretend to. A large part of a cloud bill is usage-driven — data transfer, requests, autoscaling — and cannot be known before anything runs. What the gate settles is the committed cost: the part that follows from the specification, such as instance type and size, storage, region, and redundancy. This is where the big, avoidable overruns live — the dev environment sized like production, the resource created in a loop — and it is settled before the money is spent.

Data transfer is the common objection, so it is worth a plain answer. The volume cannot be known upfront, but the volume is rarely what causes a surprise. The surprise is usually the design: a database in one region and its consumer in another, tiers split across zones, traffic sent through a NAT gateway, public egress from a resource that should have stayed private. Each of these is a choice, and each is written into the configuration the agent generates. These patterns are easy to miss in a manual review, and the agent is better placed to catch them: it generated the configuration and can see the topology it created. The gate does not price the transfer. For the patterns policy knows to watch — a cross-region dependency, egress that should be private — it raises the flag and asks whether it is intended. It will not catch every case, but it catches the ones worth a rule. The rest — the cost that shows up only in use — is caught by the monitoring in Section 7.

One more distinction matters, because IAI covers physical hardware as well as cloud. In the cloud, cost is pay-as-you-go and priced per unit, so the gate can read it from the specification. On-premise hardware is different: it was bought once, as capital, and its cost is spread over its life. A new workload on a server that already exists adds no new bill — what it consumes is a share of that amortized cost and some of the capacity left before the next hardware refresh. Physical infrastructure has no monthly charge to estimate, so the gate does not try to. The question there is different: does the workload fit the capacity already in place, and does it bring the next hardware refresh forward. IAI can only answer that if the capacity and refresh dates are recorded — and that usually sits in asset and finance systems, not in the infrastructure config. Where it is available, the gate weighs capacity and amortization instead of a per-hour rate. Where it is not, it does not guess a price; it marks the change as a capital decision for a person to make.

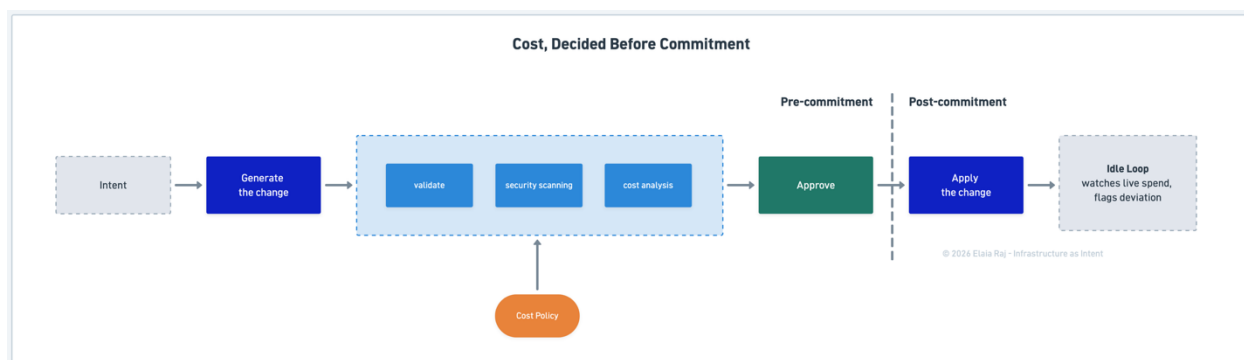


Figure 1. The cost architecture. Every change follows one path: the agent prices it and checks it against cost policy before commitment, a human approves, and only then is it applied. After commitment, the idle loop watches live spend and catches what the gate could not foresee, while the manifest ties every dollar to the reason a resource exists.

5. Spend Judged by What Matters

The second paper used load-bearing dependencies for security. The same idea applies to cost, because cost cannot be judged well without knowing what it is for.

The simple approach treats every dollar the same: cheaper is always better. That is wrong, and it is why blunt cost-cutting often does damage. A high bill on the database behind the payment service may be right — the resource is load-bearing, and the redundancy and performance that drive its cost are what its importance requires. The same bill on a side resource may be pure waste. The number is identical; the judgement is opposite; the difference is what depends on it.

Ordinary cost tools struggle here, the same way ordinary scanners struggle with risk: they see the resource but not its place. They can say a resource is expensive. They cannot easily say whether the expense is justified, because that depends on importance, which the cost tool does not hold. The intent layer holds both, because the manifest holds both. So the agent can:

- separate justified spend on load-bearing infrastructure from waste on side resources, and point attention where high cost and low importance meet;
- apply different cost limits to different levels of importance;
- describe a cost in the only terms that let a decision be made well — not just that it costs a certain amount, but that it costs that amount to serve something of a given importance.

6. Spend Tied to Its Purpose

To manage cost per product, team, or customer, every dollar has to be tied to the thing it pays for. Most organizations do this with tags — labels added to resources by hand. Tags are unreliable: they go missing, get the wrong value, or are never updated when a resource is reused, which is why a large share of cloud spend cannot be traced to anything [4].

IAI does not depend on tags. When the agent creates a resource, it records why — what the resource is for. That reason is the link between the spend and the purpose, and it is captured when the resource is created, not added afterwards. So the cost of any product or team is simply the sum of the resources created to serve it. Showback and chargeback become a direct read instead of a guess, and because the record is kept over time, it also becomes a history to forecast from.

7. Watching Cost Continuously

The cost gate governs cost at the moment of change. But much dangerous spend happens between changes, as resources bill quietly or scale with use. The idle loop is the second line.

When no change is pending, the agent watches live spend. Cost monitoring is not new — most mature organizations have it. The difference is who is watching, and what they can do about it.

A standalone cost tool watches from the outside. It can flag that spend has moved, but it does not know what the resource is for or who owns it, so it raises an alert and waits for someone to work

it out. The idle loop is run by the agent that creates and approves changes. It already holds the cost, the importance, and the owner of each resource. So when spend moves, it can route the alert to the team that owns it — not only to finance — and it can propose the fix, not just the alert: here is the spend, the likely cause, the change that would correct it, and what that change would save.

This turns cost from something reviewed each month into something watched all the time. Between changes the agent compares real spend against what was expected, and is ready to act the moment it drifts.

It is fair to ask whether this competes with the cost tools the cloud providers already ship. It does not. AWS, Google, and Azure all offer budgets, anomaly detection, and, more recently, assistants that explain spend and even cap it; AWS has opened a billing interface for agents to build on, and Google has added project-level spend caps and a cost-explanation agent [8]. These are good sources of live spend and provider pricing, and IAI uses them as inputs. What they do not do is sit on the path a change takes before it is provisioned, or hold the reason a resource exists, and each is tied to a single cloud. IAI reads their signals, normalizes across providers, and ties the number to the intent and the owner. It does not replace the native tools; it adds the decision they are missing — the one made before the change is provisioned.

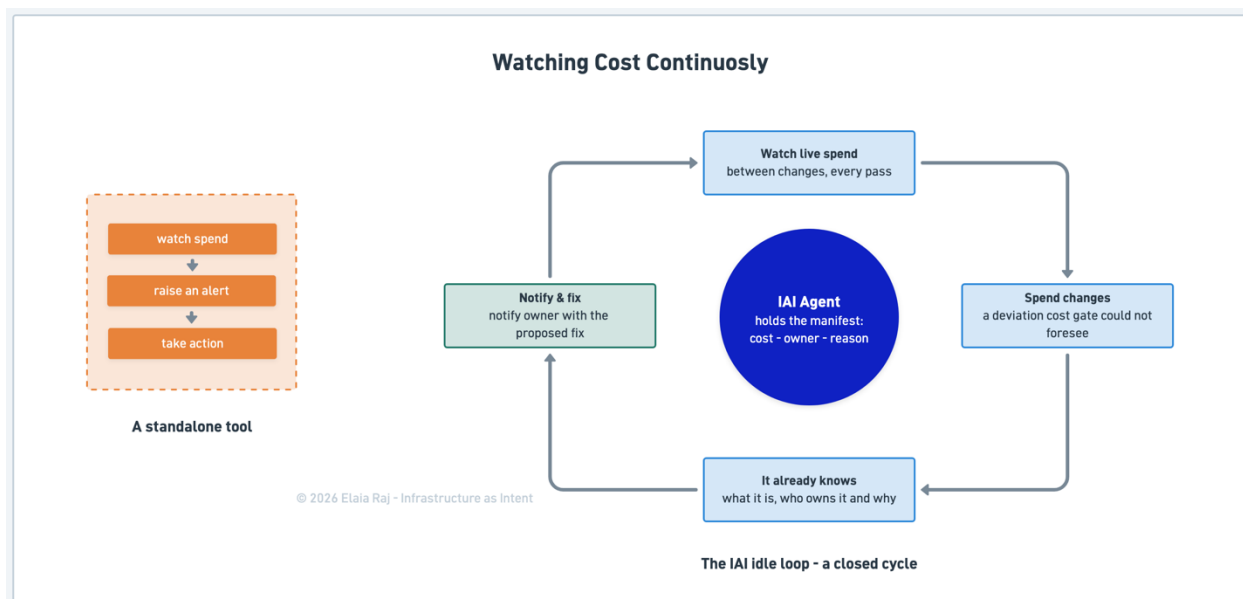


Figure 2. A standalone tool ends at the alert, leaving a person to work out what moved and who owns it. The idle loop is run by the agent that made the changes, so it already knows the cost, owner, and reason for each resource — it closes the cycle by routing to the owning team and proposing the fix.

8. Preventing the Accidental Overrun

The overrun finance fears most is the bill shock — a cost that lands as a multiple, not a margin, of what was meant. It has two sources, and IAI's two lines cover both.

The first is the overrun built into the change itself: the test environment sized like production, the resource created in a loop. These can be seen ahead of time, because the cost follows from the specification. The cost gate is the defense — the cost is worked out before apply, found to break policy, and stopped before money is committed.

The second is the overrun that only shows up in use: the AI job or usage-driven workload that scales with no ceiling, the data transfer whose cost grows with traffic. These cannot be fully seen ahead of time, and they are exactly the workloads the field finds hardest to forecast [6]. The idle loop is the defense — the agent watches real spend, catches the drift early, and proposes the correction while it is still small.

Both are bounded by policy: the limits the organization sets are enforced at the gate on every change and watched in the loop on every pass. Within the earned-trust model from the first paper, the agent may in time be allowed to act on the clearest cases on its own — pausing a runaway process or rightsizing a plainly oversized resource within set limits — while the larger or irreversible decisions stay with a human.

9. Current Industry Position

Most of the pieces exist, but they sit apart. Cost-estimation tools can price a change from its specification — but they run as an optional check beside a pipeline, not as a required gate on the only path the infrastructure changes through. Cost-anomaly tools are good — but they watch from the outside, raise alerts that have to be routed and investigated, and cannot fix what they find. Allocation and showback tools exist — but they depend on the same hand-applied tags whose unreliability is the problem [4]. And there is wide agreement that cost should be governed earlier and tied to value [1][7] — but agreement is not architecture.

What is missing is not a feature. It is one intent layer that every change passes through — one place that costs every change before apply, judges spend by what depends on it, ties every dollar to the intent it serves, watches spend between changes, and stops the accidental overrun on both fronts. The parts are on the shelf. Putting them on the one path the infrastructure changes through is what IAI does.

10. Conclusion

Cloud financial management grew up cleaning waste after the fact, and the easy gains there are mostly taken. The field is turning to governing cost, forecasting it, and tying it to value — deciding cost well rather than tidying it later [1].

IAI is built for that. It moves the estimate ahead of the commitment, so cost is governed at provisioning instead of found at month-end. It judges spend by what depends on it, so cost is weighed against importance. It ties every dollar to the intent it serves, so allocation is right by default. It watches spend between changes and acts before a runaway grows. And it closes both routes to the accidental overrun.

This is the last of the three gates. The first paper moved reasoning from people into a system. The second made security part of how infrastructure is produced. This one makes cost a decision taken before the commitment, not a number reconciled after it. In each case the gain is the same, and it is larger than speed: infrastructure whose correctness, security, and cost are built in, not recovered later.

References

- [1] FinOps Foundation. *The State of FinOps 2026 Report* (1,192 respondents representing more than \$83 billion in annual cloud spend). On the shift in practitioner priorities toward governance, forecasting, and organizational alignment, the diminishing returns of pure optimization, and AI as the top forward-looking priority. <https://data.finops.org/>
- [2] Flexera. *2026 State of the Cloud Report*. On wasted cloud spend, estimated at 29 percent in 2026 (and in the 27–32 percent range across recent years), rising again as AI workloads add cost complexity. <https://www.flexera.com/blog/finops/the-new-era-of-cloud-what-2026-data-tells-us-about-spend-scale-and-strategy/>
- [3] Spendark. "The State of Cloud Waste 2026: \$100B+ in Unnecessary Spend." On the absolute scale of wasted cloud expenditure. <https://spendark.com/blog/state-of-cloud-waste-2026/>
- [4] CloudZero. "Cloud Computing Statistics 2026." On untagged and unallocated cloud spend and the widening cost-attribution gap that makes clean allocation hard. <https://www.cloudzero.com/blog/cloud-computing-statistics/>
- [5] FinOps Foundation. *FinOps Framework — Phases: Inform, Optimize, Operate*. On the three iterative phases of the discipline and the movement of weight toward inform and operate. <https://www.finops.org/framework/phases/>
- [6] CloudZero. "FinOps for AI: Why AI Changes Cloud Cost Management." On the cost volatility of GPU and usage-driven workloads and the difficulty of forecasting AI spend. <https://www.cloudzero.com/blog/finops-for-ai/>
- [7] McKinsey & Company. "Everything is better as code: Using FinOps to manage cloud costs." On codifying financial guardrails as policy, enforced in provisioning like security checks. <https://www.mckinsey.com/capabilities/tech-and-ai/our-insights/everything-is-better-as-code-using-finops-to-manage-cloud-costs>
- [8] Google Cloud. "Introducing Spend Caps and AI Cost Visibility." On native cloud FinOps tooling — project-level spend caps, anomaly detection, and cost-explanation agents, with parallel moves from AWS and Azure. <https://cloud.google.com/blog/topics/cost-management/introducing-spend-caps-ai-cost-visibility-next26>

Infrastructure as Intent is the intellectual property of Elaia Raj. This whitepaper is part of an ongoing series.